Hafta 2 — ASP.NET Core MVC Temelleri ve Mimari Yapısı

> **Ders:** Web Programlama · **Hafta:** 2 / 13 ders haftası · **Süre:** 3 × 50 dakika **Ön koşul:** Hafta 1 (HTTP sözleşmesi, proje iskeleti, `Program.cs` 'in dört evresi) **Bu dosya sınav kapsamındadır.** Daha derin konular `H02-detay.md` dosyasındadır ve **sınav kapsamı dışıdır.**

🎯 Bu Haftanın Hedefleri

Bu haftanın sonunda:

1. **Model**, **View** ve **Controller**'ın sorumluluklarını **ayırt edebileceksiniz**; verilen bir kod parçasının hangi katmana ait olduğunu söyleyebileceksiniz.
2. Bir action metodunun neden `IActionResult` döndürdüğünü ve hangi sonuç tipinin ne zaman kullanıldığını **açıklayabileceksiniz**.
3. `/Urun/Detay/42` gibi bir adresin hangi controller ve action'a düştüğünü **çalıştırmadan belirleyebileceksiniz**.
4. Opsiyonel ve kısıtlı parametreli **route** tanımlayabileceksiniz.
5. Bir HTTP isteğinin **Middleware Pipeline**'dan nasıl geçtiğini **çizebileceksiniz**.
6. Middleware sırasının neden kritik olduğunu bir örnekle **gösterebileceksiniz**.
7. **Dependency Injection** ile bir servisi kaydedip controller'a enjekte **edebileceksiniz**.
8. `AddSingleton`, `AddScoped` ve `AddTransient` arasındaki farkı **açıklayabileceksiniz**.

🧩 **Haftanın ilkesi — Convention ve sıra**

> Çerçeveler sık yapılanı **varsayılan** yapar: sözleşmeyi bilmek, yapılandırmayı bilmekten hızlıdır. Ama aynı çerçevede **sırayı bozmak çoğu zaman hata vermez** — sessizce yanlış davranış üretir. Bu dersin en pahalı hataları buradan çıkar.

---

1. MVC Deseni

Hafta 1'de bir isteğin sunucuya kadar geldiğini gördük. Şimdi **sunucunun içinde** ne olduğuna bakıyoruz. ASP.NET Core MVC, gelen isteği işlerken kodu **üç sorumluluğa** böler.

1.1. Üç katman, üç soru

Katman	Cevapladığı soru	Tipik içeriği
Model	*Veri neye benziyor? Kuralları ne?*	`Kitap` sınıfı, fiyatın negatif olamayacağı kuralı

View	*Kullanıcı bunu nasıl görecek?*	`.cshtml` dosyası, HTML ve döngüler
Controller	*Bu istek geldiğinde ne yapılacak?*	İsteği al, gereken veriyi hazırla, bir sonuç döndür

Controller'ın işi karar vermektir, iş yapmak değil. İdeal bir action üç satırdır: veriyi al, hazırla, döndür. İş kuralları modele veya bir servise; HTML üretimi view'a aittir.

1.2. "Şişman controller" belirtisi

Bir controller metodu 50 satırı geçiyorsa, içinde muhtemelen başka katmanların işi vardır:

```
// KÖTÜ — controller hem veri erişimi hem iş kuralı hem biçimlendirme yapıyor
public IActionResult Detay(int id)
{
    var baglanti = new SqlConnection("Server=...");        // veri erişimi
    var kitap = baglanti.Query("SELECT * FROM Kitap...");   // veri erişimi
    if (kitap.Fiyat > 1000) kitap.Fiyat *= 0.9m;            // iş kuralı
    var html = "<h1>" + kitap.Baslik + "</h1>";             // HTML üretimi
    return Content(html);
}
```

Bu kodun üç sorunu var: **test edilemez**, **yeniden kullanılamaz**, ve bir şey bozulduğunda **nerede aramanız gerektiği belirsizdir.** Dönem boyunca bu üç işi üç ayrı yere taşıyacağız: veri erişimi **Hafta 8**'de EF Core'a, iş kuralı **Hafta 13**'te servis katmanına, HTML üretimi **Hafta 3**'te Razor'a.

1.3. İsteğin yaşam döngüsü

Hafta 1'deki "beş duraklı zincir"in artık **son üç durağının içini** açıyoruz:

```
İstek
  |
  ▼
[ Middleware Pipeline ]   ← §4 (bu haftanın üçüncü bloğu)
  |
  ▼
[ Routing ]               ← hangi controller, hangi action?  §3
  |
  ▼
[ Controller örneklenir ]  ← Dependency Injection burada devreye girer  §5
  |
  ▼
[ Action çalışır ]         ← §2
  |
  ▼
[ Sonuç (IActionResult) ]  ← View render edilir, ya da yönlendirme, ya da JSON
  |
  ▼
Yanıt
```

Bu şemayı aklınızda tutun: bu haftanın üç bloğu bu şemanın üç kutusudur.

1.4. Convention over Configuration (sözleşme öncelikli yapılandırma)

ASP.NET Core, **sık yapılanı varsayılan yapar.** Dosyaları belirli isimlerle belirli yerlere koyarsanız hiçbir yapılandırma yazmanız gerekmez:

Sözleşme	Anlamı
Sınıf adı `…Controller` ile biter	Bu bir controller'dır
`HomeController` → `Views/Home/`	Bu controller'ın view'ları bu klasörde aranır
`Index()` action'ı → `Index.cshtml`	`View()` çağrısı action adıyla aynı dosyayı arar
`Controllers/`, `Views/`, `Models/`	Çatının aradığı klasör adları

> ⚠️ **Sözleşmeyi bozarsanız çatı sizi bulamaz.** `Controllers` klasörünü `Denetleyiciler` yaparsanız veya sınıf adından `Controller` sonekini silerseniz, hata mesajı almazsınız — yalnızca **404** alırsınız. Bu haftanın 🐛 Hata Köşesi'nin birinci satırı budur.

---

2. Controller ve Action

2.1. Controller nedir?

`Controller` taban sınıfından türeyen, adı `…Controller` ile biten bir C# sınıfı:

```
public class UrunController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

Bu sınıf `/Urun` ve `/Urun/Index` adreslerini karşılar.

2.2. Action metotları ve `IActionResult`

Bir action metodu **ne döndürür?** Cevap: "bir sonuç". Ama sonuç HTML de olabilir, yönlendirme de, JSON da, bir dosya da. Bu yüzden dönüş tipi `IActionResult`'tır — hepsinin ortak arayüzü.

Yardımcı metot	Ne döndürür	Tipik kullanım
`View()`	HTML (Razor render edilir)	Sayfa gösterme
`View(model)`	HTML + veri	Sayfayı veriyle gösterme (**Hafta 4**)
`RedirectToAction("Ad")`	`302` + `Location` başlığı	İşlem sonrası yönlendirme
`Json(nesne)`	JSON metni	Basit API cevabı (**Hafta 12**)

`NotFound()`	`404`	Kayıt bulunamadı
`Content("metin")`	Düz metin	Hızlı deneme
`File(...)`	Dosya indirme	Rapor, ek

```
public IActionResult Detay(int id)
{
    if (id <= 0)
        return NotFound();          // 404

    return View();                  // 200 + HTML
}
```

2.3. `View()` hangi dosyayı bulur?

`return View();` yazdığınızda çatı şu sırayla arar:

```
1. Views/{ControllerAdı}/{ActionAdı}.cshtml     → Views/Urun/Detay.cshtml
2. Views/Shared/{ActionAdı}.cshtml              → Views/Shared/Detay.cshtml
```

Bulamazsa **istisna fırlatır** ve size **aradığı yolların listesini** gösterir — o hata mesajı çok işe yarar, okumayı alışkanlık edinin.

Başka bir view'ı göstermek isterseniz adını verirsiniz: `return View("Ozet");`

2.4. `[HttpGet]` ve `[HttpPost]`

Varsayılan olarak bir action **her metoda** cevap verir. Formlarla çalışırken (Hafta 4) bunu sınırlamak gerekir:

```
[HttpGet]                       // formu göster
public IActionResult Ekle() => View();

[HttpPost]                      // formu işle
public IActionResult Ekle(Kitap kitap) { /* ... */ }
```

İki metot **aynı ada** sahip olabilir çünkü HTTP metotları farklıdır. Bu, Hafta 4'teki form döngüsünün temel kalıbıdır.

2.5. `RedirectToAction` ve neden gerekli

Bir POST işleminden sonra doğrudan `View()` döndürmek **kötü bir alışkanlıktır**: kullanıcı sayfayı yenilerse form yeniden gönderilir ve ikinci kayıt oluşur. Doğrusu:

```
[HttpPost]
public IActionResult Ekle(Kitap kitap)
{
    // kaydet...
    return RedirectToAction("Index");   // 302 → tarayıcı yeni bir GET yapar
}
```

Buna **Post-Redirect-Get (PRG)** deseni denir; **Hafta 4**'te ayrıntısına bakacağız.

2.6. Action parametreleri nereden gelir?

Bir action parametresinin değeri **üç kaynaktan** gelebilir ve çatı bunları sırayla dener:

Kaynak	Örnek	Nereden okunur
Route verisi	`/Urun/Detay/42` → `id = 42`	Adresin kendisi
Sorgu dizesi	`/Urun/Ara?kelime=roman` → `kelime = "roman"`	`?` sonrası
Form gövdesi	POST edilen form alanları	İstek gövdesi (**Hafta 4**)

```
// GET /Urun/Ara?kelime=roman&sayfa=2
public IActionResult Ara(string kelime, int sayfa = 1)
{
    return Content($"'{kelime}' aranıyor, sayfa {sayfa}");
}
```

Parametre adı ile sorgu dizesi anahtarı **aynı olmak zorundadır** — `kelime` yazıp `?q=roman` göndermek işe yaramaz, `kelime` `null` gelir.

Bu mekanizmanın adı **Model Binding**'dir ve tam olarak **Hafta 4**'ün konusudur. Şimdilik bilmeniz gereken: **değer sessizce boş gelebilir**, hata almazsınız.

---

3. Routing (Yönlendirme)

Hafta 1'in sonunda şu soruyu cevapsız bırakmıştık: *adres çubuğuna hiçbir sayfa adı yazmadım, sunucu hangi kodu çalıştıracağını nereden bildi?* Cevap **routing**'dir.

3.1. Varsayılan route kalıbı

`Program.cs`'in son satırlarındaki şu kalıp her şeyi belirler:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Kalıbı parça parça okuyalım:

Parça	Anlamı
`{controller=Home}`	Birinci segment controller adıdır; **yoksa** `Home`
`{action=Index}`	İkinci segment action adıdır; **yoksa** `Index`
`{id?}`	Üçüncü segment `id` parametresidir; `?` = **opsiyonel**

Şimdi tabloyu okuyabiliriz:

Adres	Controller	Action	id
`/`	`Home`	`Index`	—
`/Urun`	`Urun`	`Index`	—
`/Urun/Detay`	`Urun`	`Detay`	—
`/Urun/Detay/42`	`Urun`	`Detay`	`42`
`/Genel/Goster`	`Genel`	`Goster`	—

> 📌 **Sınavda her yıl sorulan soru budur.** `/Genel/Goster` adresi `GenelController` **içindeki** `Goster` **action'ını** çağırır. "Home controller'ında goster" değil.

3.2. Parametre action'a nasıl ulaşır?

Route'taki `{id}` ile action parametresinin **adı aynıysa** çatı değeri kendiliğinden geçirir:

```
// GET /Urun/Detay/42
public IActionResult Detay(int id)      // id = 42
{
    return View();
}
```

`{id?}` opsiyonel olduğu için parametreyi de opsiyonel yapmak gerekir. İki yolu var:

```
public IActionResult Detay(int? id)     // id null olabilir
public IActionResult Detay(int id = 0)  // varsayılan değer
```

> ⚠️ `public IActionResult Detay(int id)` yazıp `/Urun/Detay` (id'siz) çağırırsanız `id` **0** olur — hata almazsınız. Bu, sessiz bir tuzaktır.

3.3. Route kısıtları (constraints)

Bir segmentin **tipini** zorlayabilirsiniz:

```
pattern: "{controller=Home}/{action=Index}/{id:int?}"
```

`{id:int}` → yalnızca sayı kabul eder. `/Urun/Detay/abc` artık bu route'a **eşleşmez** (404 alırsınız) — çünkü `abc` bir `int` değil.

Kısıt	Kabul eder
`{id:int}`	Tam sayı
`{id:min(1)}`	1 ve üzeri
`{ad:alpha}`	Yalnızca harf
`{ad:length(2,50)}`	2–50 karakter
`{tarih:datetime}`	Tarih

3.4. Attribute Routing (öznitelik tabanlı yönlendirme)

Varsayılan kalıp çoğu sayfa için yeter. Ama adres yapısını **elle** belirlemek isterseniz action'ın üstüne yazarsınız:

```
public class UrunController : Controller
{
    [Route("kitaplar")]                    // GET /kitaplar
    public IActionResult Index() => View();

    [Route("kitap/{id:int}")]              // GET /kitap/42
    public IActionResult Detay(int id) => View();
}
```

Ne zaman hangisi?

Durum	Yöntem
Normal sayfalar	Varsayılan kalıp — hiçbir şey yazmayın
Kullanıcı dostu / SEO adresi gerekiyor	Attribute Routing
Web API (**Hafta 12**)	Attribute Routing — orada zorunlu gibidir

3.5. Birden çok route ve eşleşme sırası

`MapControllerRoute` birden çok kez çağrılabilir. Önemli kural: **ilk eşleşen kazanır.**

```
app.MapControllerRoute(
    name: "kitapDetay",
    pattern: "kitap/{id:int}",
    defaults: new { controller = "Urun", action = "Detay" });

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

`/kitap/42` isteği **birinci** route'a eşleşir ve `UrunController.Detay(42)` çalışır. Sıralamayı ters çevirirseniz birinci route'a hiçbir istek ulaşmaz — çünkü varsayılan kalıp neredeyse her şeye eşleşir.

> ⚠️ **Kural: Özel route'lar önce, varsayılan route en sona.** Genel olan kalıbı üste koymak, altındakileri sessizce ölü koda çevirir — hata mesajı almazsınız.

3.6. URL üretimi — adresi elle yazmayın

Bir sayfaya bağlantı verirken `href="/Urun/Detay/42"` yazmak **kırılgandır**: route kalıbı değişince tüm bağlantılar sessizce bozulur. Doğrusu, çatıya adresi **ürettirmektir**:

```
<a asp-controller="Urun" asp-action="Detay" asp-route-id="42">Kitap detayı</a>
```

Bu, **Tag Helper**'dır ve **Hafta 5**'in konusudur. Şimdilik bilmeniz gereken: **adresi çatı üretir, siz yazmazsınız.**

---

4. Middleware Pipeline (Ara Yazılım Hattı)

4.1. Temel fikir

Bir istek sunucuya geldiğinde **tek bir kod parçasına** gitmez. Sıraya dizilmiş bileşenlerden **tek tek geçer.** Her bileşene **middleware** (ara yazılım) denir.

Her middleware üç şey yapabilir:

1. İsteği **görür** ve inceleyebilir
2. İsteği veya yanıtı **değiştirebilir**
3. Sıradakine **devreder** — ya da devretmeyip **short-circuit** (kısa devre) yapar



> 🧠 **Doğru benzetme:** Havaalanı kontrol noktaları. Sırayla geçersiniz; herhangi biri sizi geri çevirebilir. Pasaport kontrolünü güvenlik taramasından **önce** koyarsanız sistem "çalışır" ama yanlış çalışır.

4.2. Sıra neden kritik?

Çünkü her middleware, **kendinden öncekilerin ürettiği bilgiye** dayanır.

En somut örnek: `UseRouting` ve `UseAuthorization`.

- `UseRouting`, isteğin hangi controller/action'a gideceğine karar verir.
- `UseAuthorization`, o action'ın yetki gerektirip gerektirmediğine bakar.

`UseAuthorization`'ı `UseRouting`'den **önce** koyarsanız, yetkilendirme çalıştığı anda hedef action henüz belli değildir — dolayısıyla neyi koruyacağını bilemez.

> ⚠️ **Bu hatanın tehlikeli yanı:** Uygulama **çökmez, hata vermez.** Sayfalar açılır. Yalnızca korumanız çalışmaz. Hatayı fark etmeniz aylar sürebilir.
>
> Bu, haftanın ilkesinin tam karşılığıdır: *sırayı bozmak sessizce yanlış davranış üretir.*

Hafta 11'de Authentication'ı eklediğimizde bu sıraya geri döneceğiz.

4.3. Şablondaki sırayı okumak

```
if (!app.Environment.IsDevelopment())   // ← yalnızca ÜRETİMDE
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthorization();
app.MapStaticAssets();
app.MapControllerRoute(...);
```

Satır	Görevi
`UseExceptionHandler`	Beklenmedik hatayı yakalar, kullanıcıya düzgün bir sayfa gösterir
`UseHsts`	Tarayıcıya "bu siteye hep HTTPS ile gel" der
`UseHttpsRedirection`	`http://` isteğini `https://` ye yönlendirir
`UseRouting`	Adresi çözer, hedef action'ı belirler
`UseAuthorization`	Yetki kontrolü yapar
`MapStaticAssets`	`wwwroot` içindeki dosyaları sunar
`MapControllerRoute`	Varsayılan route kalıbını tanımlar, action'ı çalıştırır

4.4. Kendi middleware'inizi yazmak

```
app.Use(async (context, next) =>
{
    // istek sıradakine gitmeden ÖNCE
    Console.WriteLine($"→ {context.Request.Method} {context.Request.Path}");

    await next();   // sıradakine devret

    // yanıt geri dönerken SONRA
    Console.WriteLine($"← {context.Response.StatusCode}");
});
```

İki ayrıntı önemlidir:

- `await next()` çağrılmazsa **pipeline orada durur** — short-circuit olur.
- `next()`'ten sonraki kod, yanıt geri dönerken çalışır. Yani pipeline **gidiş-dönüşlüdür.**

4.5. Sıranın sessizce bozduğu ikinci örnek

`UseRouting` / `UseAuthorization` en bilinen örnektir ama tek değil. Şunu düşünün:

```
app.MapStaticAssets();     // wwwroot içindekileri sun
app.UseAuthentication();   // kim olduğunu belirle
app.UseAuthorization();    // yetkisi var mı bak
```

Statik dosya middleware'i, eşleşen bir dosya bulduğunda **short-circuit yapar** — yani dosyayı döndürür ve sıradakilere hiç uğramaz. Dolayısıyla `wwwroot` içindeki dosyalar **Authorization'a takılmadan** servis edilir.

Çoğu zaman istediğimiz budur: CSS ve logo herkese açık olmalıdır. Ama `wwwroot` altına kullanıcı faturalarını koyduysanız, o faturalar **herkese açıktır** — `[Authorize]` yazmış olmanız bir şey değiştirmez, çünkü istek oraya hiç ulaşmaz.

> 🧩 Haftanın ilkesi burada da geçerlidir: sıra yanlış olduğunda **program çalışmaya devam eder**, yalnızca güvenlik varsayımınız sessizce yanlış olur. **Hafta 13**'te dosya yükleme yaparken bu tuzağa geri döneceğiz.

---

5. Dependency Injection (Bağımlılık Enjeksiyonu)

5.1. Sorun: `new` ile nesne üretmek

Controller'ınız bir servise ihtiyaç duyuyor. En kolay yol:

```
public class KitapController : Controller
{
    public IActionResult Index()
    {
        var servis = new KitapServisi();      // ← sorun burada
        return View(servis.Hepsi());
    }
}
```

Bu kod çalışır ama üç sorunu vardır:

1. **Test edilemez** — `KitapServisi`'ni sahte bir sürümle değiştiremezsiniz.
2. **Değiştirilemez** — servisin yerine başka bir uygulama koymak için tüm controller'ları düzenlemek gerekir.
3. **Kaynak yönetimi elde değildir** — veritabanı bağlantısı gibi pahalı nesneler her istekte yeniden üretilir.

5.2. Çözüm: nesneyi çatıdan istemek

İki adım vardır. **Birinci adım:** servisi `Program.cs`'te kaydedin (ikinci evre — Hafta 1 §6.2):

```
builder.Services.AddScoped<IKitapServisi, KitapServisi>();
```

İkinci adım: controller'ın kurucusunda **isteyin**:

```
public class KitapController : Controller
{
    private readonly IKitapServisi _servis;

    public KitapController(IKitapServisi servis)   // çatı bunu doldurur
    {
        _servis = servis;
    }

    public IActionResult Index() => View(_servis.Hepsi());
}
```

Artık `new` yok. Controller "bana bir `IKitapServisi` gerek" diyor; **nereden geldiğini bilmiyor ve bilmesi de gerekmiyor.** Buna **kurucu enjeksiyonu (constructor injection)** denir.

> 💡 Hafta 1'deki şablonda bunu zaten görmüştünüz: `HomeController`, kurucusunda `ILogger<HomeController>` istiyordu. O da aynı mekanizmaydı.

5.3. Yaşam döngüleri (lifetimes)

Kayıt yaparken **nesnenin ne kadar yaşayacağını** seçersiniz:

Kayıt	Yeni nesne ne zaman üretilir	Tipik kullanım
`AddSingleton`	Uygulama başında **bir kez**	Yapılandırma, önbellek, sayaç
`AddScoped`	**Her HTTP isteği için bir kez**	Veritabanı bağlamı (`DbContext`)
`AddTransient`	**Her istendiğinde** yeniden	Hafif, durumsuz yardımcılar

`AddScoped` en sık kullanılandır: bir isteğin başından sonuna kadar aynı nesne kullanılır, istek bitince atılır. **Hafta 8**'de `DbContext`'i böyle kaydedeceğiz.

> ⚠️ **Klasik hata:** `AddSingleton` olarak kaydedilmiş bir servisin içine `AddScoped` bir servis enjekte etmek. Singleton uygulama boyunca yaşar, Scoped ise istek bitince ölür — singleton, ölmüş bir nesneyi tutmaya çalışır. ASP.NET Core bunu **çalışma zamanında yakalar** ve istisna fırlatır.

---

6. Ortam ve Yapılandırma

6.1. Geliştirme ↔ üretim

`if (!app.Environment.IsDevelopment())` satırına dikkat edin: hata işleyici ve HSTS **yalnızca üretimde** devreye girer.

Sebep: Geliştirme sırasında hatanın **tüm ayrıntısını** görmek istersiniz — hangi satır, hangi istisna, hangi yığın izi. Üretimde ise bu bilgi **kullanıcıya gösterilmemelidir**; saldırgana kodunuz hakkında bilgi verir.

Ortam, `ASPNETCORE_ENVIRONMENT` değişkeniyle belirlenir: `Development`, `Staging`, `Production`. Geliştirirken değeri `Properties/launchSettings.json` içinden gelir.

6.2. `appsettings.json`

Ayarlar buradadır ve DI ile okunur:

```
public class HomeController : Controller
{
    private readonly IConfiguration _yapilandirma;

    public HomeController(IConfiguration yapilandirma) => _yapilandirma = yapilandirma;

    public IActionResult Index()
    {
        var baslik = _yapilandirma["SiteBasligi"];
        return View();
    }
}
```

`appsettings.Development.json` yalnızca geliştirme ortamında okunur ve `appsettings.json`'daki aynı anahtarları **ezer**.

> ⚠️ **Sırlar `appsettings.json`'a yazılmaz.** O dosya kod deposuna gider. Veritabanı parolası ve API anahtarı için geliştirmede `dotnet user-secrets`, üretimde ortam değişkeni kullanılır. **Hafta 13**'te yayına alırken buna döneceğiz.

🐛 Hata Köşesi — Hafta 2

Belirti	Gerçek sebep	Çözüm
Yeni sayfa **404** veriyor	Sınıf adı `…Controller` ile bitmiyor, veya `Controllers/` dışında	Adı `UrunController` yapın, dosyayı `Controllers/` altına koyun
Action'a erişilemiyor, 404	Metot `public` değil	`public` ekleyin — `private` action'lar route'lanmaz
`InvalidOperationException: The view 'X' was not found`	View dosyası yok veya yanlış klasörde	Hata mesajındaki **aranan yollar listesini** okuyun
`/Urun/Detay` çağrılınca `id` **0** geliyor	Route'ta `{id?}` opsiyonel, parametre ise `int`	`int?` yapın veya varsayılan değer verin
`/Urun/Detay/abc` beklenmedik hata	Kısıt yok; `abc`, `int`'e çevrilemiyor	Route'a `{id:int}` kısıtı ekleyin
Servis enjekte edilemiyor: `Unable to resolve service`	`Program.cs`'te kaydedilmemiş	`builder.Services.Add…` ile kaydedin — **`Build()`'dan önce**
`Cannot consume scoped service from singleton`	Singleton içine Scoped enjekte edilmiş	Servisin yaşam döngüsünü gözden geçirin (§5.3)
Middleware eklendi ama çalışmıyor	`app.Use…` satırı **yanlış sırada** veya `MapControllerRoute`'dan sonra	Sırayı §4.3'teki listeye göre düzeltin

🕳️ Klasik Tuzak — `UseAuthorization`'ı `UseRouting`'den önce koymak

```
app.UseAuthorization();   // ← ÖNCE
app.UseRouting();         // ← SONRA
```

Bu kod **derlenir**, uygulama **çalışır**, sayfalar **açılır.** Hiçbir hata mesajı yoktur.

Tek sorun: yetkilendirme, hedefin ne olduğunu bilmeden çalışıyor. Bugün korunan sayfanız olmadığı için farkı göremezsiniz. **Hafta 11**'de `[Authorize]` eklediğinizde koruma çalışmayacak — ve yine hata almayacaksınız. Sessizce korumasız bir site yayınlayacaksınız.

Kural: `UseRouting` → `UseAuthentication` → `UseAuthorization` → `MapControllerRoute`. Bu sıra ezberlenir.

📖 Terimler Sözlüğü

Terim	Türkçe	Anlamı
Middleware	Ara yazılım	İstek pipeline'ındaki her bir bileşen
Pipeline	Boru hattı	İsteğin sırayla geçtiği middleware zinciri
Short-circuit	Kısa devre	Bir middleware'in isteği sıradakine devretmeden yanıtlaması
Routing	Yönlendirme	Adresin hangi controller/action'a düşeceğinin belirlenmesi
Route pattern	Route kalıbı	`{controller}/{action}/{id?}` biçimindeki şablon
Route constraint	Route kısıtı	Segmentin tipini zorlayan kural (`{id:int}`)
Attribute Routing	Öznitelik tabanlı yönlendirme	Adresi action'ın üstünde `[Route]` ile belirtmek
Action	—	Controller içinde bir isteği karşılayan metot
IActionResult	—	Action'ın döndürdüğü sonucun ortak arayüzü
Dependency Injection (DI)	Bağımlılık enjeksiyonu	Nesneyi `new` ile üretmek yerine çatıdan istemek
Constructor injection	Kurucu enjeksiyonu	Bağımlılığın sınıfın kurucusundan alınması
Convention over Configuration	Sözleşme öncelikli yapılandırma	Sık yapılanın varsayılan olması
Scoped / Singleton / Transient	—	Servis yaşam döngüleri (§5.3)

✅ Kontrol Soruları

A. MVC deseni

1. Model, View ve Controller'ın her birinin cevapladığı soruyu bir cümleyle yazın.
2. Aşağıdaki kod parçaları hangi katmana ait olmalı? (a) `if (fiyat < 0) throw new Exception();` (b) `<h1>@Model.Baslik</h1>` (c) `return RedirectToAction("Index");`
3. "Şişman controller" ne demektir? Üç sorununu yazın.
4. `HomeController` içindeki `Hakkimizda()` action'ı hangi view dosyasını arar? İki yol yazın.

B. Controller ve action

5. `IActionResult` neden somut bir tip değil de arayüz?
6. Şu durumlarda hangi yardımcı metodu kullanırsınız: (a) sayfa göster, (b) kayıt bulunamadı, (c) işlem sonrası başka sayfaya git, (d) JSON döndür?
7. Aynı adı taşıyan iki action metodu nasıl bir arada bulunabilir?
8. Bir POST action'ının sonunda `View()` döndürmek neden kötüdür? Doğrusu nedir?
9. Aşağıdaki kodda action `/Urun/Detay/0` ile çağrıldığında ne olur?

```
public IActionResult Detay(int id)
{
    if (id <= 0) return NotFound();
    return View();
}
```

C. Routing

10. `{controller=Home}/{action=Index}/{id?}` kalıbındaki üç parçayı ve `=` / `?` işaretlerinin anlamını yazın.
11. Aşağıdaki adresler hangi controller ve action'a düşer? `/` · `/Kitap` · `/Kitap/Ara` · `/Kitap/Detay/7` · `/Genel/Goster`
12. `/Urun/Detay` çağrıldığında `Detay(int id)` metodunda `id` kaç olur? Bu neden tuzaktır?
13. `{id:int}` kısıtı eklenirse `/Urun/Detay/abc` isteğine ne olur?
14. Attribute Routing'i ne zaman tercih ederiz? İki durum yazın.
15. Bir bağlantıyı `href="/Urun/Detay/42"` diye yazmak yerine çatıya ürettirmek neden daha iyi?

D. Middleware Pipeline

16. Bir middleware'in yapabileceği üç şeyi yazın.
17. "Short-circuit yapmak" ne demektir? Hangi middleware tipik olarak short-circuit yapar?
18. Aşağıdaki sıralamada sorun nedir ve sonucu ne olur?

```
app.UseAuthorization();
app.UseRouting();
app.MapControllerRoute(...);
```

19. Bir önceki sorudaki hatanın **en tehlikeli yanı** nedir?
20. Aşağıdaki middleware'de `await next();` satırı silinirse ne olur?

```
app.Use(async (context, next) =>
{
    Console.WriteLine("istek geldi");
    await next();
    Console.WriteLine("yanıt gidiyor");
});
```

21. `UseExceptionHandler` neden `if (!app.Environment.IsDevelopment())` bloğunun içindedir?
22. Bir geliştirici kullanıcı faturalarını `wwwroot/faturalar/` klasörüne koyuyor ve fatura sayfasındaki action'a `[Authorize]` ekliyor. Faturalar korunmuş olur mu? Cevabınızı **pipeline sırası** üzerinden gerekçelendirin.

E. Dependency Injection

23. Controller içinde `new KitapServisi()` yazmanın üç sorununu yazın.
24. `AddSingleton`, `AddScoped` ve `AddTransient` arasındaki farkı birer cümleyle açıklayın.
25. `DbContext` hangi yaşam döngüsüyle kaydedilir? Neden?
26. Singleton bir servisin içine Scoped bir servis enjekte edilirse ne olur?
27. `Unable to resolve service for type 'IKitapServisi'` hatasını alıyorsunuz. İlk nereye bakarsınız?
28. `appsettings.json` içindeki bir ayara controller'dan nasıl erişilir?

📑 Kaynakça

> Microsoft bağlantılarındaki `?view=aspnetcore-10.0` parçasını silmeyin. *Bağlantılar 21 Eylül 2026'da denetlendi.*

Zorunlu okuma

#	Kaynak	Neyi dayandırıyor
1	Get started with ASP.NET Core MVC — Part 2: Add a controller	§1–§2 — controller ekleme, action, routing ile ilk temas. **Bu haftanın okuma ödevi**
2	ASP.NET Core MVC'ye genel bakış	§1 — MVC deseni ve katman sorumlulukları

Controller, action ve routing (§2–§3)

#	Kaynak	Neyi dayandırıyor

3	Controller'larda action'ları işlemek	§2 — action metotları, `IActionResult`
4	Controller action'larına routing	§3 — route kalıbı, kısıtlar, Attribute Routing
5	ASP.NET Core'da routing (genel)	§3 — routing'in çatı düzeyindeki çalışması

Middleware (§4)

#	Kaynak	Neyi dayandırıyor
6	ASP.NET Core middleware	§4.1–§4.3 — pipeline, short-circuit, **önerilen sıra**
7	Özel middleware yazmak	§4.4 — `app.Use(async (ctx, next) => …)` kalıbı
8	Statik dosyaları sunmak	§4.5 — statik dosya middleware'inin short-circuit davranışı

Dependency Injection ve yapılandırma (§5–§6)

#	Kaynak	Neyi dayandırıyor
9	ASP.NET Core'da Dependency Injection	§5 — servis kaydı, kurucu enjeksiyonu, **yaşam döngüleri**
10	Yapılandırma	§6.2 — `appsettings.json`, `IConfiguration`
11	Ortamlar (Development / Production)	§6.1 — `ASPNETCORE_ENVIRONMENT`
12	Geliştirmede uygulama sırlarını güvenli tutmak	§6.2 — `dotnet user-secrets`

📚 Daha Fazlası

`H02-detay.md` dosyasındadır ve **sınav kapsamı dışıdır**:

- MVC deseninin tarihi: Smalltalk'tan ASP.NET Core'a
- Routing motoru içeride nasıl çalışır (endpoint, `RouteValueDictionary`)
- `IActionResult` ile `ActionResult<T>` farkı
- Middleware'i sınıf olarak yazmak (`IMiddleware`, `UseMiddleware<T>`)
- `Use` / `Run` / `Map` farkları ve dallanan pipeline
- Area (alan) kavramı ve büyük projelerde klasör düzeni
- DI kabının içi: servis konumlandırıcı karşıt deseni, `IServiceScope`
- Yapılandırma sağlayıcılarının öncelik sırası

🔜 Haftaya Hazırlık

🏠 Ev görevi

`KampusKitap` projenize:

1. **İki controller** ekleyin: `KitapController` ve `KategoriController`
2. Her birine **üçer action** yazın (örn. `Index`, `Detay`, `Ara`) — hepsi şimdilik `Content("...")` döndürebilir

3. Aşağıdaki beş adresin hangi controller/action'a düşeceğini **çalıştırmadan** bir kâğıda yazın, sonra çalıştırıp doğrulayın: `/` · `/Kitap` · `/Kitap/Detay/5` · `/Kategori/Ara` · `/Kitap/Detay/abc`
4. `Program.cs`'e §4.4'teki **günlükleyen middleware'i** ekleyin ve terminal çıktısını inceleyin
5. **Teslim:** tahmin listeniz + gerçek sonuçlar + terminal çıktısının ekran görüntüsü

İsteğe bağlı (yapana artı): `IKitapServisi` adında bir arayüz ve onu uygulayan bir sınıf yazıp `AddScoped` ile kaydedin, `KitapController`'a enjekte edin.

Okuma: Kaynakça #1 — *Add a controller* (Part 2)

Haftaya (Hafta 3 — Razor, View Component, Layout): Bu hafta action'lar `Content()` ile düz metin döndürdü. Hafta 3'te **HTML üretmeye** geçiyoruz: Razor sözdizimi, her sayfayı saran Layout ve tekrar eden parçaları tek yerden yönetmek.

Düşünerek gelin: `Views/Shared/_Layout.cshtml` dosyasını açın. İçinde `@RenderBody()` diye bir satır var. Sizce ne yapıyor?